LAB PROGRAM

7. Python Program to Implement Breadth First Search (BFS)

Aim

To implement the **Breadth First Search (BFS)** algorithm in Python to traverse all the vertices of a graph level by level.

Algorithm

1. Create a graph using an adjacency list.
2. Choose the starting node.
3. Create an empty queue and a visited set.
4. Add the starting node to both the queue and visited set.
5. While the queue is not empty:
    - Remove the front node from the queue.
    - Display the node.
    - Visit all unvisited adjacent nodes.
    - Add them to the queue and mark them as visited.
6. Stop when all reachable nodes are visited.

Objective

- To traverse all the nodes of a graph using the **Breadth First Search (BFS)** algorithm.
- BFS visits nodes **level by level** using a **queue**.
- Ensure every node is visited only once.

Pseudo Code

START

Create graph

Choose start node

Create empty queue

Create visited set

Enqueue start node

Mark start node as visited

WHILE queue is not empty

    Dequeue a node

    Print the node

    FOR each adjacent node

        IF node is not visited

            Mark as visited

            Enqueue node

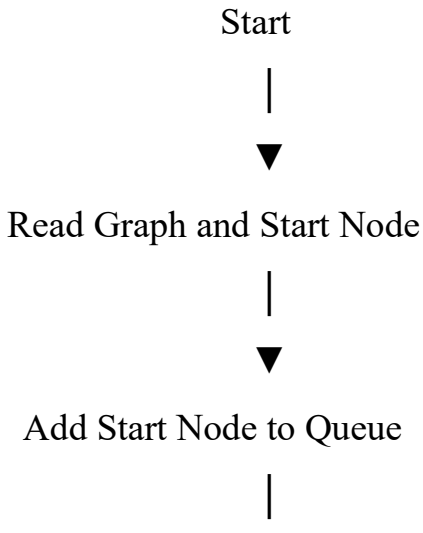
        END IF

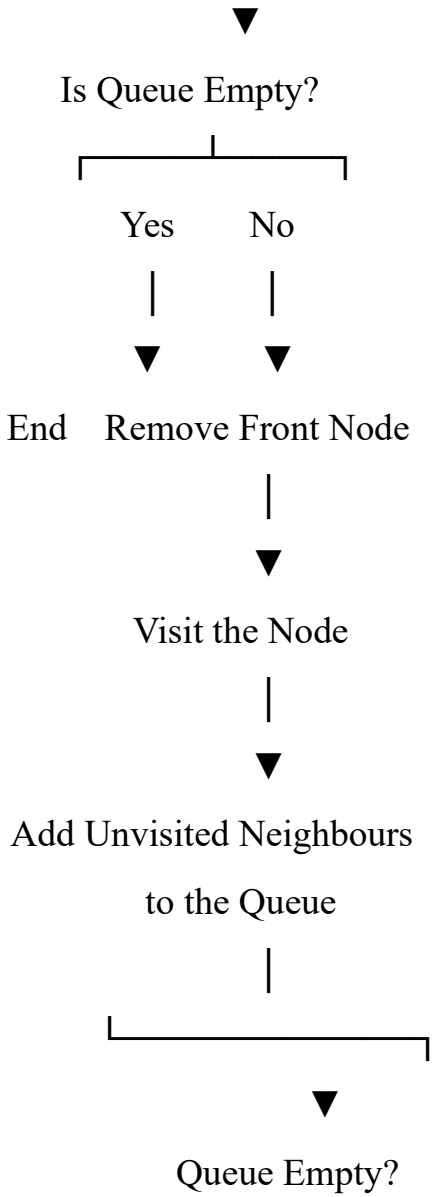
    END FOR

END WHILE

STOP

Flowchart:



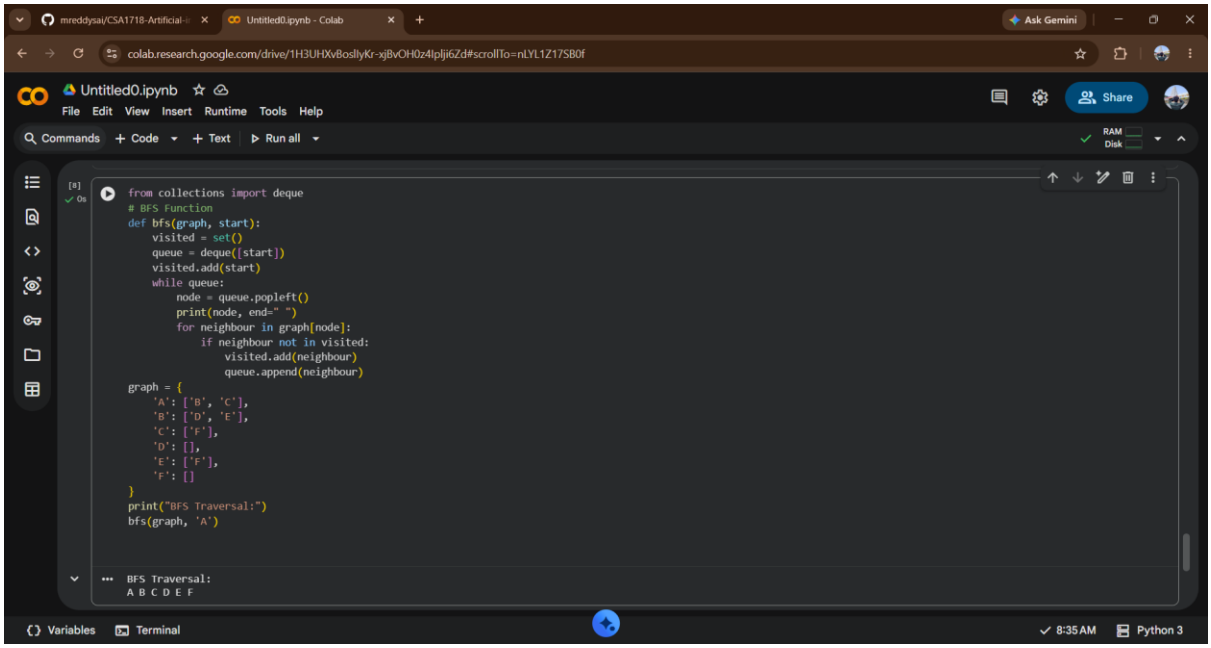


Python Code

```
from collections import deque
# BFS Function
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    while queue:
        node = queue.popleft()
```

```
        print(node, end=" ")
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
print("BFS Traversal:")
bfs(graph, 'A')
```

Output



Result

The **Breadth First Search (BFS)** algorithm was successfully implemented in Python, and the graph was traversed in **level-by-level order** starting from the given source node.

8. Python Program to Implement Depth First Search (DFS)

Aim

To implement the **Depth First Search (DFS)** algorithm in Python to traverse all the vertices of a graph using depth-first traversal.

Algorithm

1. Create a graph using an adjacency list.
2. Choose the starting node.
3. Create an empty visited set.
4. Visit the starting node and mark it as visited.
5. Print the current node.
6. Recursively visit each unvisited adjacent node.
7. Repeat until all reachable nodes are visited.

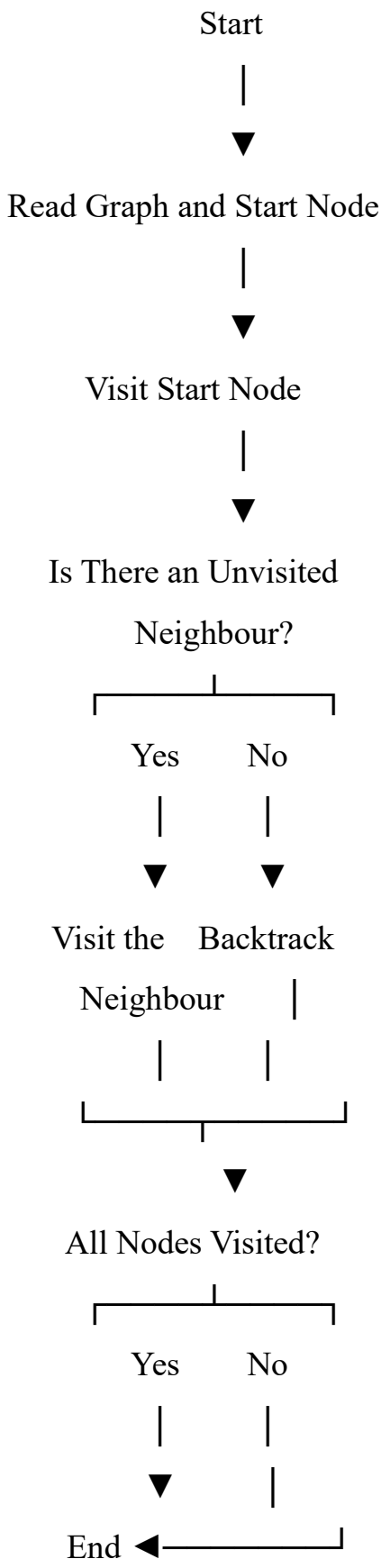
Objective

- To traverse all the nodes of a graph using the **Depth First Search (DFS)** algorithm.
- DFS explores one path completely before moving to another path.
- It uses **recursion** (or a stack) to visit each node only once.

Pseudo Code

```
START
Create graph
Choose start node
Create empty visited set
FUNCTION DFS(node)
   Mark node as visited
   Print node
   FOR each adjacent node
      IF adjacent node is not visited
         DFS(adjacent node)
      END IF
   END FOR
END FUNCTION
Call DFS(start node)
STOP
```

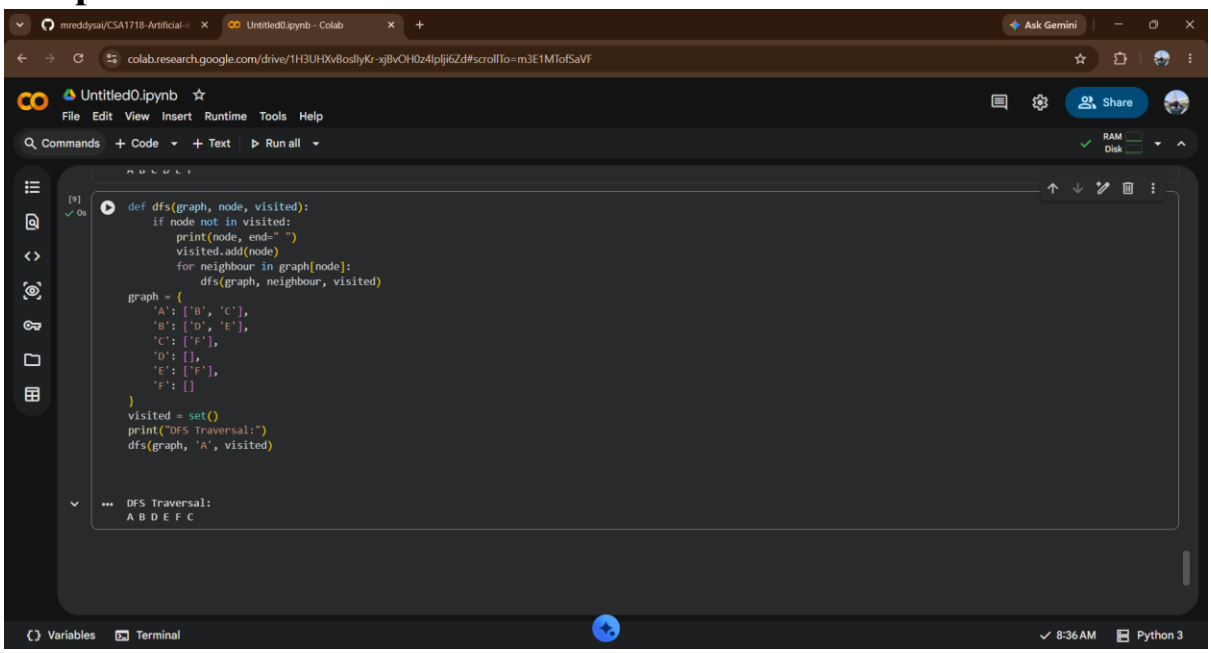
Flowchart:


Start
Read Graph and Start Node
Visit Start Node
Is There an Unvisited Neighbour?
Yes
No
Visit the Neighbour
Backtrack
All Nodes Visited?
Yes
No
End

Python Code

```
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(graph, neighbour, visited)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
visited = set()
print("DFS Traversal:")
dfs(graph, 'A', visited)
```

Output :



Result

The **Depth First Search (DFS)** algorithm was successfully implemented in Python, and the graph was traversed in **depth-first order** starting from the given source node.